CAMPUSFIX

A Role-Based College Complaint and Maintenance Management System

Demo and Viva Guide

Fourth-Year CSE Mini Project

Student Name: [Student Name]

Roll Number: [Roll Number]

Guide Name: [Faculty Guide Name]

Academic Session: 2026-27

Department of Computer Science and Engineering

How to Use This Guide

This guide prepares a non-coding presenter for a clear five-to-seven-minute demonstration and the most likely technical questions. Practise the sequence once before the review, keep the browser zoom at 100 percent, and explain the reason for each role change rather than clicking quickly.

Local Setup

Open PowerShell in the CampusFix project folder and run:

```
.\.venv\Scripts\python.exe manage.py migrate

.\.venv\Scripts\python.exe manage.py create_demo_engagement

.\.venv\Scripts\python.exe manage.py runserver 127.0.0.1:8000 --noreload
```

Then open http://127.0.0.1:8000/. Keep the command window open during the demonstration.

Local Demo Accounts

LOCAL DEMO ONLY - never reuse these passwords for a real deployment.

Role	Username	Password
Student	student_demo	CampusFixStudent@2026
Maintenance Staff	staff_demo	CampusFixStaff@2026
Administrator	admin_demo	CampusFixAdmin@2026

Five to Seven Minute Demo Script

Time	Screen	What to say and show
0:00-0:35	Landing page	Introduce the problem: informal maintenance complaints have no shared status, ownership, or audit trail.
0:35-1:25	Student	Log in as student_demo. Show the dashboard, open Report an issue, point out category, location, urgency, evidence, and server-owned ticket fields. Open CF-2026-00005 and show priority, status, timeline, and feedback.
1:25-2:35	Administrator	Log out and sign in as admin_demo. Show live dashboard totals. Open the complaint queue, filter if useful, open CF-2026-00001, and explain review, assignment, rejection, and the permanent timeline.
2:35-3:35	Maintenance Staff	Log in as staff_demo. Show that only assigned tickets appear. Open CF-2026-00003 and explain that Assigned can be started and In Progress can be resolved with a meaningful note.
3:35-4:20	Student follow-up	Return to student_demo. Open notifications and the resolved ticket. Explain that workflow updates are private and that feedback is allowed once after resolution or

Time	Screen	What to say and show
		closure.
4:20-5:35	Analytics and reports	Return to admin_demo. Show analytics charts and accessible tables. Open Reports, combine a filter, point out totals, staff workload, feedback, CSV downloads, and print view.
5:35-6:15	Security proof	As Student, open /dashboard/administrator/ and show the branded 403 page. Explain that hiding a link is not the security control; the server checks the role.
6:15-6:45	Close	State the verified result: 84 automated tests passed, system checks are clean, and the project meets both proposal and implementation review criteria.

Major Features in Plain English

Authentication

Django checks identity using a username, securely stored password hash, and session cookie.

Role permissions

Every protected page checks whether the signed-in account is a Student, Staff member, or Administrator.

Complaint ticket

One record stores the problem, place, urgency, status, priority, assignment, and dates.

Workflow

Only allowed next steps can happen, so a ticket cannot jump unpredictably between states.

Timeline

Every workflow action adds a permanent event instead of replacing the past.

Notifications

The system creates private updates for the specific account affected by an action.

Feedback

The reporting Student can give one rating after work is completed.

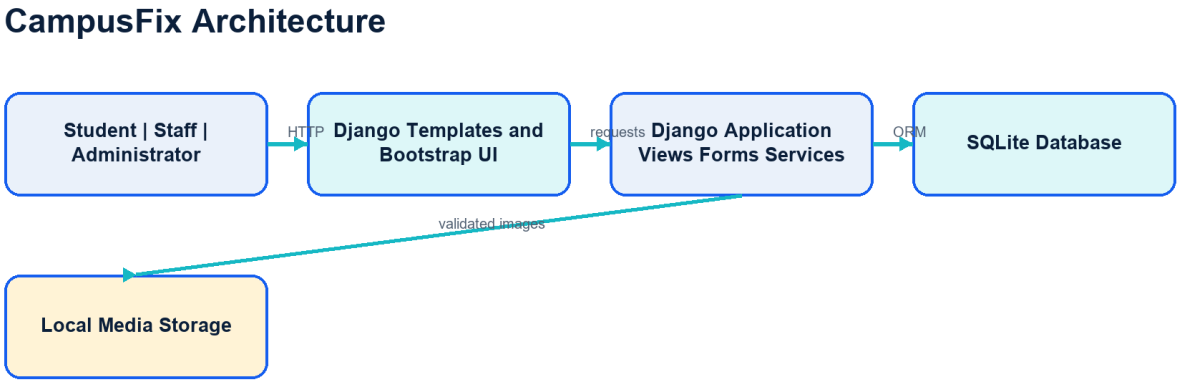
Analytics

Database counts are grouped into charts and accessible tables.

Reports

Administrator filters are reused for totals, table rows, print output, and CSV exports.

Architecture in Plain English



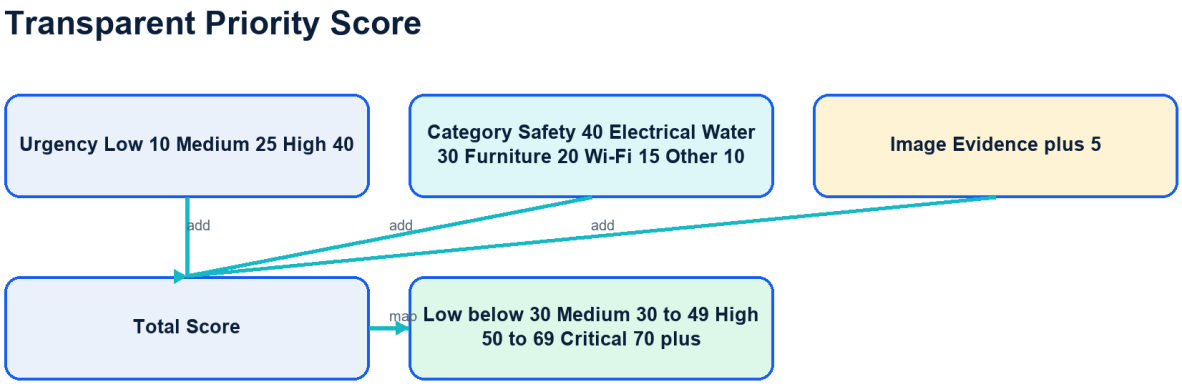
CampusFix architecture

The browser displays HTML pages. Django receives requests, checks permissions, validates forms, runs workflow rules, and reads or writes records through its ORM. SQLite stores structured data and the media folder stores validated images. Everything runs locally for the academic demonstration.

Database Models in Plain English

Model	Simple explanation
User	A person who signs in, with one CampusFix role and profile details.
ComplaintCategory	A reusable type such as Electrical, Water, or Safety.
CampusLocation	A reusable campus place such as a lab or library.
Complaint	The main ticket containing the reported issue and current workflow state.
ComplaintHistory	One permanent event in the ticket timeline.
Notification	A private update addressed to one user.
ComplaintFeedback	The Student's one-to-one rating and optional comment for completed work.

Priority Scoring in Plain English



Transparent complaint priority calculation

CampusFix adds urgency points, category points, and five points for valid image evidence. It maps the total to Low, Medium, High, or Critical and stores a written explanation. This is transparent triage, not artificial intelligence.

Permissions Notifications and CSV Safety

Role checks happen on the server. Students can see only their own complaints, Staff only assigned complaints, and Administrators the management and reporting pages. Notifications are account-specific and deduplicated. CSV safety prefixes values beginning with =, +, -, or @ so spreadsheet software treats them as text rather than executable formulas.

Likely Viva Questions and Answers

1. What problem does CampusFix solve?

It replaces informal maintenance reporting with traceable tickets, controlled assignments, status updates, and reports.

2. Why did you choose Django?

Django provides authentication, forms, ORM, templates, admin, migrations, security middleware, and testing in one framework.

3. Why is the project monolithic?

The scope is small and cohesive. One deployable Django project is easier to understand, test, and demonstrate than distributed services.

4. What is authentication?

Authentication proves who the user is, normally through login credentials and a session.

5. What is authorisation?

Authorisation decides what an authenticated user is permitted to view or change.

6. How is role-based access implemented?

The custom User has a role field, and server-side decorators plus ownership or assignment checks protect views.

7. Why is hiding navigation links insufficient?

A user can type a URL directly. The server must reject an unauthorised request even when the link is hidden.

8. What are the three roles?

Student, Maintenance Staff, and Administrator.

9. How are ticket numbers generated?

After creation, the complaint receives CF, the creation year, and a zero-padded database sequence.

10. What is the complaint lifecycle?

Submitted, Under Review, Assigned, In Progress, Resolved, and Closed, with Rejected as a controlled terminal path.

11. Where are workflow rules stored?

In central service functions so pages cannot implement conflicting transition logic.

12. Why use database transactions?

They keep the complaint update and its history event consistent if an operation fails.

13. What is the timeline?

ComplaintHistory rows record action, previous status, new status, actor, note, and time.

14. Can a Staff member resolve any complaint?

No. The server verifies that the complaint is assigned to that active Staff account.

15. Can a Student edit a submitted ticket?

No. The submitted record is read-only to preserve accountability.

16. How are uploads validated?

The form limits size to 5 MB and Pillow decodes content, verifies JPEG, PNG, or WebP, and the model uses a random filename.

17. How is priority calculated?

Urgency points plus category points plus five points for image evidence; thresholds map the score to four levels.

18. Is priority artificial intelligence?

No. It is a deterministic, explainable rule designed for this project.

19. How do notifications work?

Workflow functions create private Notification rows for affected users, using dedupe keys to avoid repeats.

20. When can feedback be submitted?

Only by the complaint owner after the ticket is Resolved or Closed, and only once.

21. How are analytics created?

Django ORM aggregation counts complaints by status, category, urgency, and priority and Chart.js displays the data.

22. Why are tables shown with charts?

Canvas charts are not fully available to screen readers, so the same values appear in HTML tables.

23. What filters do reports support?

All time, 30 or 90 days, custom dates, status, category, priority, urgency, assigned Staff, and search.

24. How is pagination handled?

The server returns 20 report rows per page and keeps the active query filters in page links.

25. What is CSV formula injection?

Spreadsheet software may execute a cell beginning with formula characters. CampusFix prefixes those values with an apostrophe.

26. Why use SQLite?

It is transactional, serverless, easy to reproduce, and suitable for a local academic application.

27. What would change for production?

Use PostgreSQL, managed media, HTTPS, secure environment variables, backups, monitoring, and a production web server.

28. How is CSRF handled?

Django middleware validates a secret token included in every state-changing POST form.

29. What testing was performed?

The final Batch 6 suite ran 84 automated tests plus checks, migrations, static collection, and real HTTP role smoke tests.

30. What does the 403 page prove?

It proves the server understood the request but denied it because the account lacked permission.

31. What is an ORM?

It lets Python code work with database records as model objects while Django generates parameterised SQL.

32. What is a migration?

A version-controlled description of a database schema change that Django can apply consistently.

33. Why have demo commands?

They produce repeatable reference data and role accounts without manual database editing and are restricted to DEBUG mode.

34. What is the main limitation?

The current local SQLite and media setup is not designed for multi-server production deployment.

35. What is the most important design decision?

Centralising workflow transitions and enforcing permissions on the server protects integrity across every page.

36. How does the project satisfy Review 1?

It defines a real problem, measurable objectives, related work, methodology, and justified tools.

37. How does it satisfy Review 2?

It provides a working role-based demonstration, completed features, test evidence, diagrams, and explainable technical decisions.

Last Minute Revision Sheet

Remember	One-line answer
Purpose	Track campus maintenance from report to verified completion.
Stack	Python, Django 6.1.1, SQLite, Bootstrap, Chart.js, Pillow.
Roles	Student reports; Administrator controls; assigned Staff performs work.
Security	Server-side roles, ownership, assignment, CSRF, password validation, safe uploads.
Lifecycle	Submitted -> Review -> Assigned -> In Progress -> Resolved -> Closed; optional Rejected.
Priority	Urgency + category + image evidence, with stored explanation.
Evidence	Permanent timeline, notifications, feedback, analytics, reports.
Tests	84 passing automated tests at the end of Batch 6.
Production future	PostgreSQL, cloud media, HTTPS, monitoring, email/SMS, mobile app.

Common Demo Mistakes

Mistake	Prevention or recovery
Server is not running	Start it before opening the browser and keep PowerShell open.
Using the wrong role	Read the username before login; log out between role changes.
Changing the prepared ticket too early	Use the listed demo sequence and show existing records before performing actions.
Showing a password on screen	Type credentials without narrating or exposing a password manager.
Trying an invalid transition	Explain the current status and use only the action displayed by CampusFix.
Losing the page after logout	Return to /login/ and continue with the next demo account.
Charts are slow to appear	Wait briefly for Chart.js; the accessible data tables are still valid evidence.
A question is unclear	Repeat the question, state the related feature, then answer with one concrete implementation fact.

Final Presentation Checklist

- Replace [Student Name], [Roll Number], and [Faculty Guide Name] in the report and presentation.
- Run the demo preparation command once and confirm all three accounts can log in.
- Keep browser zoom at 100 percent and close unrelated tabs or windows.
- Do not claim cloud deployment, email/SMS, a mobile app, or machine learning.
- End with the verified facts: clean checks and 84 passing tests.